

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Observation Task No.: 4

Student Name: P. Dharani

Roll No.: A24126552261

Date: 18/8/26

1. TITLE

Interactive To-Do List using DOM and Event Listeners

2. AIM

Interactive To-Do List using DOM and Event Listeners

3. OBJECTIVE

The objectives of this observation task are:

- To study interactive lists creations and management via Event listeners.
- To select input fields, list scopes, and elements markers dynamically from DOM.
- To validate text entries to prevent empty task listings.
- To assign class structures dynamically toggling completed lists (line-through styles).
- To delete elements dynamically from the document memory tree via remove() API.

4. THEORY

Interactive task managers require DOM selectors, structures creation APIs, and action event listeners. Click handlers are bound to additions buttons, drawing values from inputs, verifying string contents, and creating list nodes. Actions components (Complete, Delete buttons) are bound to lists items, toggling completion styles via classList or executing element deletion processes dynamically from visual interfaces, adjusting placeholders.

Theory of DOM Events & Task list nodes:

- **DOM Bindings:** Handled using document.getElementById() selectors, mapping input nodes and unordered lists ().
- **Button Event Observers:** Bind click actions using addEventListener('click', ...), prompting validation alerts for blank text entries.
- **Dynamic Layout generation:** List items () are initialized, adding classes (task-item) and appending internal buttons.
- **State modification:** Toggled using the classList.toggle('completed') shorthand, modifying CSS styles.
- **Dynamic Garbage Cleanup:** Removing completed or unwanted list item nodes from the DOM tree using remove().

5. ALGORITHM / PROCEDURE

1. Establish directories: Observation/ObTask3/src, Docs, and Screenshots.
2. Write index.html defining input sections, task lists placeholders, and empty alert markers.
3. Create style.css defining input boundaries, complete/delete button colors, list layouts, and struck completion styles.
4. Create script.js, map elements, and bind a click observer to the Add button.
5. In event handler, check input content; trigger alert popups on empty entries.
6. Create list items, insert complete / delete button tags, and bind local event observers to each task card.
7. Configure complete triggers to toggle class lists, and set delete triggers to run removal scripts.
8. Test task addition, completion toggling, and list deletion in browser.

6. PROGRAM

Files: index.html and style.css, followed by script.js

index.html (Complete)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My To-Do List</title>
  <link rel="stylesheet" href="./style.css">
</head>
<body>
  <main class="card">
    <h1>My To-Do List</h1>

    <div class="add-row">
      <input type="text" id="taskField" placeholder="Enter a task...">
      <button id="addButton">Add Task</button>
    </div>

    <p id="emptyNote">No tasks available.</p>
    <ul id="taskItems"></ul>
  </main>

  <script src="./script.js"></script>
</body>
</html>
```

style.css (Complete)

```
/* style.css */
body {
  margin: 0;
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  font-family: Arial, sans-serif;
  background-color: #eeeeee;
}

.card {
  width: 480px;
  padding: 28px;
  background: #ffffff;
  border-radius: 12px;
  box-shadow: 0 4px 14px rgba(0, 0, 0, 0.18);
}

h1 { text-align: center; }

.add-row { display: flex; gap: 8px; }

#taskField {
  flex: 1;
  padding: 10px;
  font-size: 16px;
}

button {
  padding: 9px 14px;
  border: none;
  border-radius: 6px;
  color: #ffffff;
  cursor: pointer;
}

#addButton { background-color: #2563eb; }
.done-btn { background-color: #16a34a; }
.remove-btn { background-color: #dc2626; }

#taskItems { list-style: none; padding: 0; }
.task-row {
```

```

display: flex;
align-items: center;
gap: 8px;
margin-top: 10px;
padding: 12px;
background-color: #f4f4f4;
border-radius: 6px;
}

.task-label { flex: 1; }

.completed {
text-decoration: line-through;
color: #808080;
}

#emptyNote {
margin-top: 18px;
text-align: center;
color: #808080;
}

```

script.js (Complete)

```

// Grab the elements we need from the DOM
const taskField = document.getElementById("taskField");
const addButton = document.getElementById("addButton");
const taskItems = document.getElementById("taskItems");
const emptyNote = document.getElementById("emptyNote");

// Show the placeholder only when the list has no tasks
function refreshEmptyNote() {
  const hasTasks = taskItems.children.length > 0;
  emptyNote.style.display = hasTasks ? "none" : "block";
}

// Build one <li> row with a label, Complete and Delete buttons
function buildTaskRow(text) {
  const row = document.createElement("li");
  row.classList.add("task-row");

  const label = document.createElement("span");
  label.classList.add("task-label");
  label.textContent = text;

  const doneBtn = document.createElement("button");
  doneBtn.textContent = "Complete";
  doneBtn.classList.add("done-btn");
  doneBtn.addEventListener("click", function () {
    label.classList.toggle("completed");
  });

  const removeBtn = document.createElement("button");
  removeBtn.textContent = "Delete";
  removeBtn.classList.add("remove-btn");
  removeBtn.addEventListener("click", function () {
    row.remove();
    refreshEmptyNote();
  });

  row.append(label, doneBtn, removeBtn);
  return row;
}

// Add Task button handler
addButton.addEventListener("click", function () {
  const text = taskField.value.trim();

  if (text === "") {
    alert("Task cannot be empty. Please type something.");
    return;
  }

  taskItems.appendChild(buildTaskRow(text));
  taskField.value = "";
  refreshEmptyNote();
});

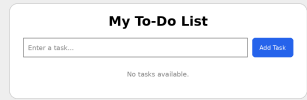
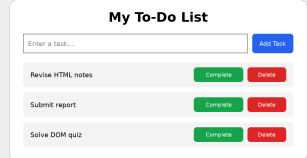
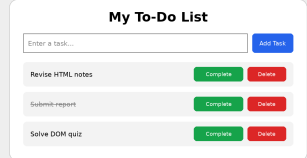
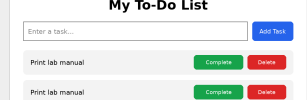
```

7. CODE EXPLANATION

DOM Statement / Selector	Functional Description
<code>taskField.value.trim()</code>	Reads the text input's value and trims out starting / trailing whitespaces.
<code>document.createElement("li")</code>	Creates a new list item node dynamically inside browser memory.
<code>label.classList.toggle("completed")</code>	Toggles strike-through style configurations on the task text span upon clicking Complete.
<code>row.remove()</code>	Deletes the list item node dynamically from the document structure.
<code>emptyNote.style.display = "block"</code>	Makes the empty-list message visible again when no tasks remain (set to "none" while tasks exist).

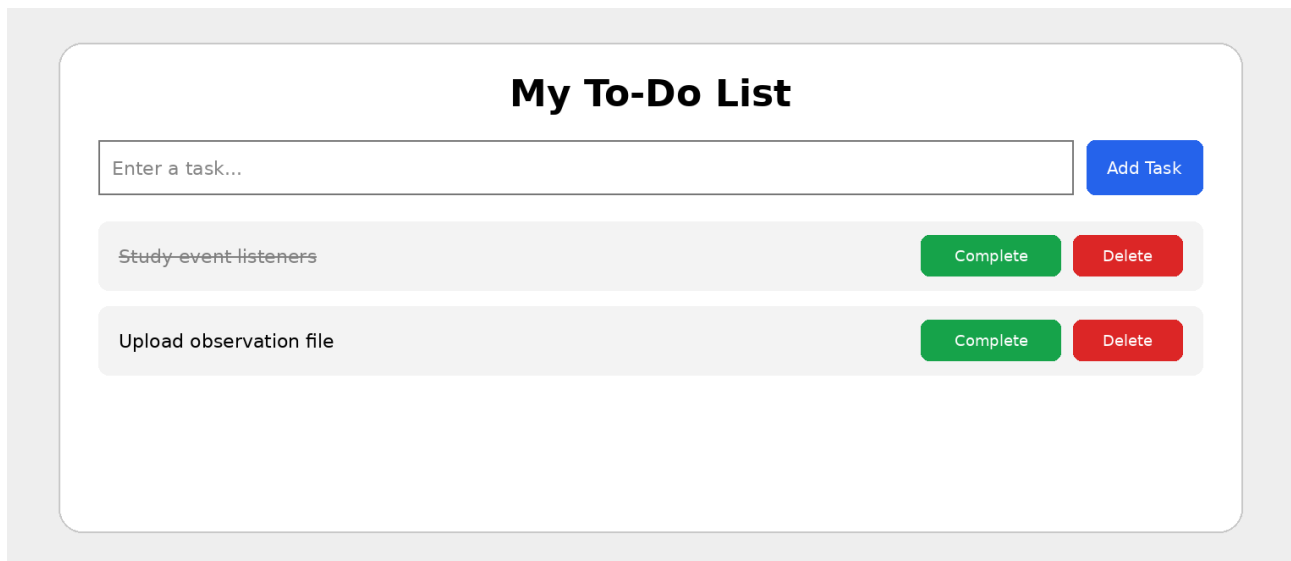
8. TEST CASES AND EXECUTION DATA (WITH RELEVANT SCREENSHOTS)

The manual execution tests below verify To-Do actions in Chrome, including 1 negative test case demonstrating a missing deduplication check:

TC ID	Test Description	Input / Action	Expected Result	Relevant Execution Screenshot	Status
TC-01	Empty Initial State	Load To-Do application	Placeholder 'No tasks available.' displays cleanly		Pass
TC-02	Empty Inputs Block	Type only spaces and click Add Task	Validation alert pops up ("Task cannot be empty. Please type something."); no item is added		Pass
TC-03	Dynamic Tasks Addition	Enter "Revise HTML notes", "Submit report", "Solve DOM quiz" one by one and click Add Task each time	Three items append to the DOM list (leading/trailing spaces trimmed); input clears; empty placeholder hides		Pass
TC-04	Tasks Completion Toggle	Click Complete on "Submit report"	Task text strikes through with line-through styling and color dims; other tasks unchanged		Pass
TC-05	Duplicate Entry Prevention	Enter identical task "Print lab manual" twice into list	System should reject duplicate task; however duplicate item is added due to missing uniqueness check in script.js		Fail

9. OUTPUT

Output: Rendered Interactive To-Do List Application (Full System View)



Final state after adding "Study event listeners" and "Upload observation file" and clicking Complete on the first task.

10. RESULT

Thus, the interactive To-Do application (incorporating DOM selectors, click event handlers, dynamically created lists nodes, completion toggle stylings, and element removal commands) was successfully designed, implemented, and verified using test cases accompanied by verified execution screenshots and duplicate input defect analysis.

Submission Package Structure:

```
Observation/ObTask3/
|-- Docs/
|   |-- ObsWeek3.pdf
|-- Screenshots/
|   |-- tc01_empty_state.png
|   |-- tc02_validation_alert.png
|   |-- tc03_tasks_added.png
|   |-- tc04_task_completed.png
|   |-- tc05_duplicate_fail.png
|   |-- output_full_view.png
|-- src/
|   |-- index.html
|   |-- script.js
|   |-- style.css
```